

Roadmap completo Programador Backend

Tudo que você precisa estudar para se tornar um desenvolvedor backend profissional — organizado em 6 fases.

Este roadmap cobre todos os tópicos fundamentais e avançados da carreira de backend. Siga as fases em ordem — fundamentos fracos criam problemas sérios nas etapas avançadas. Tempo estimado: **12 a 18 meses** de estudo consistente.

■ Fase 1 — Fundamentos

Obrigatório

Lógica de programação

Base de tudo. Sem isso, nada funciona.

Variáveis · Condicionais · Loops · Funções · Recursão

Estruturas de dados

Como organizar e manipular dados eficientemente.

Arrays · Listas · Pilhas · Filas · Hash maps · Árvores · Grafos

Algoritmos

Soluções eficientes para problemas comuns.

Busca · Ordenação · Complexidade $O(n)$ · Divisão e conquista

Como a internet funciona

Entender o que acontece por baixo dos panos.

TCP/IP · DNS · HTTP/HTTPS · Portas · Request/Response

Terminal / Linha de comando

Você vai viver no terminal. Aprenda cedo.

Bash · Navegação · Permissões · Processos · Pipes

Git e controle de versão

Indispensável em qualquer time ou projeto.

Commits · Branches · Merge · Rebase · Pull requests

■ Fase 2 — Linguagem de programação

Escolha uma

Python

Fácil de aprender, muito usado em APIs, dados e IA.

Tipos · OOP · Async · FastAPI · Django

JavaScript / Node.js

Uma linguagem para front e back. Grande ecossistema.

ES6+ · Promises · Express · NestJS · TypeScript

Java

Muito usado em sistemas corporativos e bancários.

OOP · Spring Boot · Maven/Gradle · JVM

Go (Golang)

Performance alta, ótimo para serviços escaláveis.

Goroutines · Channels · Gin · Interfaces

■ *Para iniciantes: comece com Python ou Node.js. Depois aprenda os fundamentos de orientação a objetos, tipagem e programação funcional.*

■■ Fase 3 — Banco de dados

Obrigatório

SQL

A língua dos bancos relacionais. Essencial.

SELECT · JOIN · Índices · Transações · Views · Stored procedures

Bancos relacionais

Para dados estruturados com relações entre tabelas.

PostgreSQL · MySQL · Normalização · ACID

Bancos NoSQL

Para dados não estruturados, flexíveis ou de alta escala.

MongoDB · Redis · Cassandra · DynamoDB

ORM

Camada entre código e banco, sem escrever SQL na mão.

Prisma · TypeORM · SQLAlchemy · Hibernate

Modelagem de dados

Saber desenhar o esquema antes de criar as tabelas.

ER diagrams · Normalização · 1FN–3FN · Relações

APIs REST

O padrão mais usado para comunicação entre sistemas.

Verbos HTTP · Status codes · CRUD · Versionamento · HATEOAS

GraphQL

Alternativa ao REST com queries flexíveis.

Queries · Mutations · Subscriptions · Schema · Resolvers

Autenticação e autorização

Controle de quem acessa o que no seu sistema.

JWT · OAuth 2.0 · Sessions · Roles · Cookies · SSO

Cache

Velocidade. Não busque no banco o que já foi buscado.

Redis · Memcached · CDN · Cache headers · Invalidação

Filas de mensagem

Processe tarefas pesadas fora do ciclo da requisição.

RabbitMQ · Kafka · SQS · Workers · Event-driven

WebSockets / Tempo real

Para chat, notificações e dados que atualizam ao vivo.

WebSocket · SSE · Socket.io · Long polling

Arquitetura e padrões

Como estruturar o código para ser sustentável.

MVC · Clean Arch · SOLID · Design Patterns · DDD

Microservices vs Monolito

Escolha a arquitetura certa para o tamanho certo.

Monolito modular · Microservicos · API Gateway · Service mesh

Docker

Empacote sua aplicação para rodar em qualquer lugar.

Containers · Dockerfile · Docker Compose · Volumes · Networks

Linux

Servidores rodam Linux. Você precisa se virar nele.

SSH · systemd · Logs · Permissões · Cron · Processos

Cloud (AWS / GCP / Azure)

Onde sua aplicação vai viver em produção.

EC2 · S3 · Lambda · RDS · Load Balancer · IAM

CI/CD

Automatize testes e deploys. Código em produção mais rápido.

GitHub Actions · GitLab CI · Jenkins · Pipelines

Kubernetes (k8s)

Orquestração de containers em escala. Avançado mas poderoso.

Pods · Services · Deployments · Ingress · Helm

Monitoramento

Saiba o que acontece no seu sistema em tempo real.

Logs · Métricas · Tracing · Prometheus · Grafana · Datadog

■ Fase 6 — Segurança e boas práticas

Não ignore

Segurança em APIs

Sua API vai ser atacada. Esteja pronto.

OWASP Top 10 · SQL Injection · XSS · CSRF · Rate limiting

HTTPS e criptografia

Trafegue dados de forma segura entre sistemas.

TLS/SSL · Certificados · Hashing · bcrypt · Criptografia simétrica

Testes automatizados

Garanta que seu código funciona antes de ir a produção.

Unitários · Integração · E2E · TDD · Mocks · Coverage

Performance e escalabilidade

Seu sistema precisa aguentar crescer.

Profiling · N+1 queries · Lazy loading · Horizontal scaling · CDN

Documentação

Quem vai usar sua API sem saber como ela funciona?

Swagger / OpenAPI · README · Postman Collections

Fases	Conteúdo	Tempo estimado
1 – 2	Fundamentos + Linguagem	3 a 6 meses
3 – 4	Banco de dados + APIs	4 a 6 meses
5 – 6	Infraestrutura + Segurança	Contínuo na carreira

■ **Dica final:** não tente aprender tudo de uma vez. Escolha uma linguagem, construa um projeto real (uma API simples com banco de dados), e vá adicionando camadas conforme a necessidade aparece. **Projeto real > curso infinito.**